

CS120 Course Project Report

李子健(SIST), 杨秉熹(SIST)

Abstract This project series focuses on two core technical directions: acoustic data transmission based on audio signal processing and network protocol emulation based on virtual network devices. The entire project is divided into four interconnected sub-projects, with each sub-project building on the achievements of the previous one and gradually increasing in technical difficulty and functional complexity.

Project 1 lays the foundation for acoustic bit transmission by introducing phase-shift keying (PSK) and frame-based coding; Project 2 optimizes the acoustic transmission scheme with Manchester coding and adds reliable transmission mechanisms such as ACK and CSMA; Project 3 further extends the system to the network layer by encapsulating acoustic transmission modules into network-capable components and implementing multi-node communication and IP packet forwarding; Project 4 completes the upper-layer protocol stack by implementing the TCP three-way handshake to achieve reliable end-to-end network communication.

This report provides a detailed and in-depth expansion of each project's implementation principles, technical details, and key considerations, aiming to provide a complete and operable implementation guide for the entire project series. All sub-projects adhere to the principle of “debuggability first”, with MATLAB-based audio waveform observation as a core debugging tool to ensure the feasibility and robustness of the implementation.

1. Project 1: Acoustic Link

Project 1 serves as the foundational sub-project of the entire series, whose core goal is to realize the reliable transmission of long bit streams through acoustic signals. Unlike traditional acoustic communication schemes that rely on chirp signals for frame synchronization, this project adopts a more controllable line coding frame header and phase-shift keying (PSK) for data modulation, with a strong emphasis on debuggability and transmission stability.

1.1 Core Technical Means and Implementation Details

1.1.1 MATLAB-Based Audio Waveform Observation: The Core Debugging Tool

A critical priority in Project 1 (and all subsequent acoustic-related sub-projects) is the use of MATLAB for audio waveform observation and analysis, which is regarded as a “printf-like” debugging capability for acoustic communication systems.

In the specific implementation process:

- The transmitter exports the generated modulated audio signal as a standard audio file (e.g., WAV format) with a fixed sampling rate (typically 48kHz)
- MATLAB is used to read this audio file and plot the time-domain waveform
- For the received audio signal, the receiver saves the recorded audio data as a WAV file, then uses MATLAB for waveform visualization, time-domain segmentation, and frequency-domain analysis

This MATLAB-based debugging method has two core values:

1. Allows developers to directly observe whether the shape of the modulated audio signal conforms to theoretical expectations
2. Provides a basis for troubleshooting transmission errors

1.1.2 Phase-Shift Keying (PSK) and Carrier Signal Design

Project 1 adopts Phase-Shift Keying (PSK) as the core modulation technique. The carrier signal is defined as a standard sine wave:

$$c(t) = A \cdot \sin(2\pi f_c t + \varphi)$$

where:

- A = amplitude
- f_c = carrier frequency (1kHz to 4kHz)
- t = time variable
- φ = initial phase (core parameter for PSK)

In Binary Phase-Shift Keying (BPSK):

- Bit 0: $s_0(t) = A \cdot \sin(2\pi f_c t)$ (phase 0°)
- Bit 1: $s_1(t) = A \cdot \sin(2\pi f_c t + \pi) = -A \cdot \sin(2\pi f_c t)$ (phase 180°)

1.1.3 Line Coding Frame Header: 10100101

Project 1 adopts a fixed line coding frame header: the 8-bit binary sequence “10100101”.

Advantages:

- Unique bit pattern not easily found in data streams
- Balanced number of 0s and 1s (4 each)
- Consistent with line coding format of data bit stream

1.1.4 Implementation Platforms: Python (sounddevice) and C++ (ASIO)

Python implementation (using sounddevice):

- “Whole recording first, then post-analysis” method
- Used for early debugging and non-real-time scenarios
- Easy to implement and debug

C++ implementation (using ASIO):

- “Recording while parsing” method
- Low-latency audio processing (millisecond level)
- Core platform for real-time requirements

1.1.5 Bit Decision: Only Extract the Middle Part of Each Bit

To improve decoding accuracy, the receiver only extracts the middle part of each bit signal segment for decision.

Implementation steps:

1. Determine start and end positions of each bit segment
2. Discard 10-20% of data at start and end
3. Retain middle 60-80% as valid decision data
4. Perform phase/amplitude detection on valid data

1.1.6 Long Packet Segmentation and CRC Packet Check

Long packet segmentation strategy:

- Divide long bit streams into fixed-length small packets (64, 128, or 256 bits)
- Insert silent intervals between packets (2-5 times bit duration)
- Transmit packets one by one in order

CRC whole packet verification:

- Packet structure: “frame header (8 bits) + valid data (N bits) + CRC (M bits)”
- CRC polynomial example: CRC-16-CCITT ($x^{16} + x^{12} + x^5 + 1$)
- Receiver recalculates CRC and compares with received value

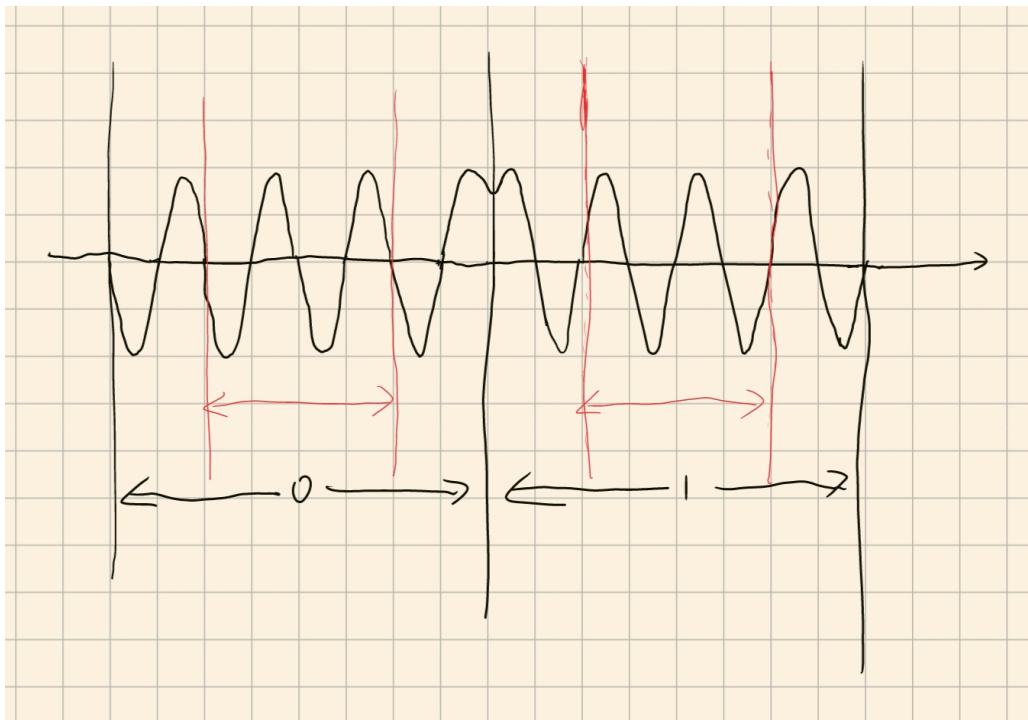


Figure 1: Figure 1: Waveform Comparison of Bit Segments (Edge Distortion vs. Middle Valid Part)

2. Project 2: Manage Multiple Access

Project 2 further optimizes the acoustic transmission scheme and adds reliable transmission mechanisms.

2.1 Core Technical Optimizations: Manchester Coding

Manchester coding is a self-clocking binary coding method:

- Bit 1: Signal transitions from high to low in middle of bit period
- Bit 0: Signal transitions from low to high in middle of bit period

Advantages:

- Strong clock self-synchronization capability
- Balanced DC component
- Resists clock drift between transmitter and receiver

2.2 Real-Time Implementation Based on C++ ASIO

Optimizations:

1. **ASIO buffer size optimization:** Set to integer multiple of bit duration
2. **Callback function optimization:** Modular design with clear responsibilities
3. **Transmitter-receiver integration:** Single device with full-duplex capability

2.3 ACK Mechanism: Reliable Transmission with Retransmission and Sliding

Window

2.3.1 Basic ACK Implementation Process

Sender side:

1. Divide bit stream into small packets with sequence numbers
2. Send packet and start timeout timer
3. If ACK received within timeout: send next packet
4. If timeout: retransmit packet (max 3-5 retries)

Receiver side:

1. Receive and verify packet (CRC check)
2. If packet expected: store data and send ACK
3. If duplicate packet: discard but resend ACK

2.3.2 Sliding Window Mechanism

- Send window size (W_s): Maximum unacknowledged packets sender can send
- Receive window size (W_r): Maximum out-of-order packets receiver can buffer
- Default: $W_s = 4$, $W_r = 1$ (adjustable)

2.4 CSMA Mechanism: Multi-Device Coordination with Parameter Tuning

CSMA (Carrier Sense Multiple Access) implementation:

1. **Carrier sense:** Detect amplitude of received acoustic signal
2. **Backoff mechanism:** Random backoff time when channel busy
3. **Collision detection:** Monitor amplitude during transmission

Key parameter tuning:

1. ASIO buffer size: Balance between latency and data loss
2. Backoff time: $>$ ASIO buffer processing time
3. Carrier detection threshold: 2-3 times environmental noise amplitude
4. Collision detection threshold: 1.5-2 times sent signal amplitude

3. Project 3: To the Internet

Project 3 extends the system to the network layer.

3.1 Preparatory Work: Wireshark-Based Ping Packet Analysis

Use Wireshark to analyze ping program packets:

- IP packet structure
- ICMP encapsulation in IP packets
- Packet sequence numbers and timestamps
- Round-trip process between nodes

3.2 Virtual Network Device Configuration

Options:

1. **Microsoft Loopback Adapter:** Virtual network adapter for Windows
2. **Closed Wi-Fi network:** Local network without internet access

Core requirement: Device must be monitorable by Scapy.

3.3 Encapsulation of Project 2 into Network-Capable Modules

Encapsulation steps:

1. **Module interface design:** Standard APIs (`send_packet()`, `receive_packet()`, etc.)
2. **Packet encapsulation:** Add data link layer header to IP packets
3. **Packet decapsulation:** Extract IP packets from data link layer packets
4. **Integration with Python:** Compile module as Python extension

3.4 Multi-Node Communication and IP Packet Forwarding

Communication scenarios:

1. Node1 → Node2: direct communication
2. Node3 → Node1: across-port communication
3. ~~Node1 → Node3 across-port communication~~
4. Node1 → 1.1.1.1: NAT
5. Node4 → Node1: Use specific payload
6. Node4 → Node1 (Fragment): IP packet fragmentation/reassembly

Key functions:

- **IP packet forwarding:** Maintain routing tables
- **IP fragmentation/reassembly:** According to IP protocol specifications

4. Project 4: TCP Handshake Implementation for Upper-Layer

Reliable Communication

Project 4 implements TCP three-way handshake for transport layer communication.

4.1 TCP Segment Structure

Includes: source port, destination port, sequence number, acknowledgment number, control bits (SYN, ACK, FIN), window size, checksum, etc.

4.2 Three-Way Handshake Implementation

1. **First handshake (SYN):** Client sends SYN with initial sequence number ISN_c
2. **Second handshake (SYN+ACK):** Server responds with SYN+ACK, acknowledgment = ISN_c+1
3. **Third handshake (ACK):** Client sends ACK, acknowledgment = ISN_s+1

4.3 Integration with Lower-Layer Module

- TCP module calls network layer API from Project 3
- Maintain connection control block (CCB) for each TCP connection

5. Additional Suggestion: Network Security Project Extension

Based on the complete protocol stack, add network security extensions:

ARP spoofing attack:

1. Attacker sends forged ARP responses to target and gateway
2. Target thinks attacker's MAC = gateway's MAC
3. Gateway thinks attacker's MAC = target's MAC
4. All traffic forwarded through attacker (man-in-the-middle)

Implementation steps:

1. Use Scapy to construct forged ARP packets
2. Implement ARP spoofing in virtual network environment
3. Use Wireshark to analyze attack traffic
4. Study defense methods (static ARP, detection tools, port security)

Conclusion

This report provides a detailed expansion of the four sub-projects of acoustic and network communication. The project series starts with acoustic data transmission and gradually extends to network and transport layers, forming a complete communication system. Each sub-project has clear goals, detailed implementation plans, and key considerations. MATLAB-based waveform observation and Wireshark-based protocol analysis serve as core debugging tools. Parameter tuning in CSMA, module encapsulation, and TCP handshake implementation are key challenges requiring extensive debugging and optimization. Through this project series, developers can master core technologies in acoustic signal processing, network protocol emulation, and reliable communication.